# Persistent Interaction Nets: A Monotone Foundation for Datalog-Family Logic

Victor Borja

*vborja@apache.org*

**Abstract.** We introduce **Persistent Interaction Nets** (PΔ-nets), an interaction system derived from Salvadori's Δ-Nets (Salvadori 2025) by replacing the consuming rewrite rule with a monotone accumulation rule. Where Δ-Nets reduces a net toward a normal form by consuming active pairs, PΔ-nets accumulates derived agents at a fixpoint without consuming their progenitors. We establish a formal correspondence — the **Levels-Strata Theorem** — between the Δ-Nets level and level-delta system and the stratum function of a locally stratified Datalog program, and we prove that the translation realizing this correspondence is **injective** with a complete structural characterization of its image. We show that the Δ-Nets status machine (unpaired / unknown / resolved) is a structural realization of the three-valued well-founded semantics (WFS; Van Gelder, Ross & Schlipf 1991) — true, false, undefined — and we prove the **full biconditional**: a replicator is unpaired at fixpoint if and only if its atom lies on an unresolved negative cycle. Building on this, we prove the paper's former central conjecture as a **theorem for all programs**, including those with negative cycles: the PΔ-net fixpoint of a translated program equals its well-founded model. We further prove that the first evaluation phase computes exactly the semi-naive Datalog fixpoint within a stratum. We establish five core properties of PΔ-nets: **monotonicity**, **stratification**, **monotone confluence**, **full parallelism**, and **module-layer optimality**. We then address optimal evaluation of pure functional-logic languages: pure-Nix module evaluation is precisely a locally stratified Datalog computation, and a hybrid PΔ-net / consuming Δ-net system achieves both module-level semi-naive optimality and Lévy-optimal expression reduction. Finally we show that CUE's constraint semantics (Hăvărneanu 2026) provides an independent derivation of PΔ-nets' Monotone Confluence theorem via finite-basis saturation, and we scope out failure propagation and incremental retraction as layering concerns that require no extension of the core calculus.

## 1. Introduction

### 1.1 The fundamental mismatch between Δ-Nets and Datalog

Salvadori's Δ-Nets is a consuming rewrite system. Its reduction relation transforms an active pair — two agents connected via their principal ports — by removing both agents and installing a result subgraph (Salvadori 2025, §3). The meaning of a Δ-net is its unique normal form, reached when no active pair remains.

Datalog-family logic with well-founded semantics (WFS; Van Gelder, Ross & Schlipf 1991) has the opposite computational character. Derivation is monotone: facts only accumulate, never retract. The meaning of a Datalog program is its **well-founded model** — the unique three-valued fixpoint of the monotone consequence operator, extended to handle negation via the alternating fixpoint (Van Gelder 1989) or SLG tabling (Chen & Warren 1996). No fact is ever consumed by deriving another.

This is not a surface difference. In Δ-Nets, active pairs are consumed because the rewrite rules represent **substitution**: the argument is placed inside the function, consuming both. In Datalog, a rule $h :- b_1, b_2$ does not consume $b_1$ or $b_2$ when deriving $h$; they remain available for other rules.

The question of this paper: is there a formulation of interaction-net-style agents and wiring that supports monotone accumulation while preserving the structural machinery — levels, level deltas, the status machine — that makes Δ-Nets analytically powerful?

### 1.2 The shared problem: computing without redundancy

Despite the semantic opposition, both systems solve the **same core problem**: computing a result that depends on shared subcomputations without performing any computation twice.

In Δ-Nets, optimal reduction (Salvadori 2025, §7) — no reduction operation that is rendered unnecessary later is applied, and no necessary reduction is applied more than once — is achieved via replicators and the level system. Sharing a λ-abstraction among $n$ uses requires a replicator with $n + 1$ auxiliary ports; its level and level-delta annotations ensure each sharing step fires exactly once.

In Datalog, tabling (SLG; Chen & Warren 1996) achieves the same property: each subgoal is solved once, memoized, and shared among all callers without re-derivation. The fundamental problem — share-don't-repeat — is identical. Replicators and tabled goals are the same concept at different abstraction levels.

### 1.3 Contributions

All results below are now established as theorems with complete proofs.

1. **Persistent Interaction Nets** (§2): a formal definition of the P$\Delta$-net system with five proven core properties — monotonicity, stratification, monotone confluence, full parallelism, and module-layer optimality.
2. **Levels-Strata Theorem and injectivity** (§3): for any locally stratified Datalog program $D$ with stratum function $s$, the translation $T(D)$ produces a P$\Delta$-net in which the level of each agent equals the stratum of the corresponding relation (Theorem 3.1); the translation $T$ is injective up to relabeling, with a complete structural characterization of its image (Theorem 3.4); and the ground-term encoding is settled canonically (Theorem 3.5).
3. **Status Machine as Well-Founded Semantics** (§4): the $\Delta$-Nets status machine is a structural realization of WFS three-valuedness, and the correspondence is a **biconditional** — a replicator is unpaired at fixpoint iff its atom lies on an unresolved negative cycle (Theorem 4.3).
4. **Fixpoint = WFS Model** (§5): the P$\Delta$-net fixpoint of $T(D)$ equals the well-founded model $M(D)$ for **all** Datalog programs, not only locally stratified ones (Theorem 5.1).
5. **Phase structure and semi-naive evaluation** (§6): the first evaluation phase computes exactly the semi-naive Datalog fixpoint within a stratum (Theorem 6.1).
6. **CUE semantics as external validation** (§7): CUE's finite-basis saturation theorem (Hăvărneanu 2026) is an independent derivation of P$\Delta$-net Monotone Confluence.
7. **Optimal evaluation of pure functional-logic languages** (§8): a P$\Delta$-net system, hybridized with consuming $\Delta$-Nets for expression bodies, optimally evaluates pure-Nix, with precise statements of which optimality notion applies to which layer.

## 2. Persistent Interaction Nets

### 2.1 Background: $\Delta$-Nets agents

We inherit the three agent types of $\Delta$-Nets verbatim (Salvadori 2025, §3.1):

- **Fan**: one principal port, two auxiliary ports. The fan has no level or deltas.
- **Eraser**: one principal port, zero auxiliary ports.
- **Replicator**: one principal port, $n+1$ auxiliary ports ($n \in \mathbb{N}$). Each auxiliary port carries an integer **level delta** $d_i \in \mathbb{Z}$. The replicator carries a non-negative integer **level** $l \in \mathbb{N}$.

Each agent has exactly one principal port and any number of auxiliary ports. Wires connect ports. An **active pair** is two agents whose principal ports are connected. Every port of any node in a proper net is either a child port or a parent port, and every wire connects a child port to a parent port (Salvadori 2025, §3.2).

We inherit the **polarity / pairedness** distinction (Salvadori 2025, §3.2): replicators start as **fan-in** (principal = child port, auxiliaries = parent ports); fan replication produces **fan-out** replicators (principal = parent port, auxiliaries = child ports).

We inherit the **status machine** (Salvadori 2025, §5.3):

- Replicators start as **unpaired**.
- When an unpaired replicator interacts with a fan, both resulting replicators become **unknown**.
- An unknown replicator $B$ consecutive to an unpaired replicator $A$, with $0 \leq l_B - l_A \leq d$ ($d$ = connecting delta), is determined **unpaired**: no agent can interact with $B$ before $A$ annihilates; since $A$ is unpaired it never annihilates, so $B$ cannot either.

### 2.2 The persistent firing rule

**Definition 0.1**: (P$\Delta$-net). A persistent interaction net is a tuple $N = (A, W, \sigma)$ where:

- $A$ is a finite multiset of agents, each carrying a type (fan / eraser / replicator$(l, d_0 ... d_n)$) and a status $\sigma(a) \in \{\text{unresolved}, \text{resolved}\}$;
- $W$ is a set of wires connecting ports;
- an **unresolved active pair** is a pair $(a, b) \in A^2$ connected via principal ports with $\sigma(a) = \sigma(b) = \text{unresolved}$.

**Definition 0.2**: (persistent firing rule). For an unresolved active pair $(a, b)$:

1. mark $\sigma(a) :=$ resolved, $\sigma(b) :=$ resolved;
2. add to $A$ the result agents $R(a, b)$ that Δ-Nets would install as a replacement;
3. connect the new agents' external wires to the same endpoints as in the Δ-Nets rule, but attached to fresh wire stubs rather than to $a$, $b$;
4. do **not** remove $a$ or $b$ from $A$.

The result agents $R(\alpha, \beta)$ are identical to those specified in Δ-Nets' interaction rules. Only the removal of the progenitors is suppressed.

**Remark.** A resolved agent's principal port remains wired to its former partner's principal port, but the pair is no longer active. Resolved agents participate in wiring but not in further firing. An unresolved agent can form new active pairs only with other unresolved agents.

**Definition 0.3**: (fixpoint). $N$ is at **fixpoint** when no unresolved active pair exists. We write $F(N)$ for the fixpoint reached from $N$.

### 2.3 Core properties

**Theorem 0.4** (Monotonicity): The agent multiset is non-decreasing under persistent firing: $|A(N')| \geq |A(N)|$ for any $N'$ reachable from $N$.

*Proof*: By Definition 2.2, step 2 only adds agents to $A$ and step 4 forbids removal. Hence every firing weakly increases $|A|$. □ □

**Theorem 0.5** (Full Parallelism): Any set of pairwise distinct unresolved active pairs in $N$ may fire simultaneously without conflict.

*Proof*: Each firing marks exactly the two agents of its pair as resolved and adds fresh agents. By the interaction-net one-principal-port constraint, each agent participates in at most one active pair at a time, so distinct pairs share no agent. Simultaneous firing of disjoint pairs therefore writes to disjoint agents and adds disjoint fresh agents; the result is identical to any sequential interleaving. There is no write conflict. □ □

**Theorem 0.6** (Monotone Confluence): The PΔ-net reduction relation is Church-Rosser: any two reduction sequences from the same initial net $N$ reach the same fixpoint $F(N)$.

*Proof*: The agent multiset $A$ is monotone non-decreasing (Theorem 2.1). Each distinct active pair $(a, b)$ produces a fixed result set $R(a, b)$ by the deterministic interaction rules, and fires at most once: once both agents are resolved the pair is ineligible. Define the accumulation operator $T_N(S) = S \cup \{\text{result of each unresolved active pair in } S\}$ over the complete lattice of agent multisets ordered by inclusion. $T_N$ is monotone (adding agents can only create more active pairs, never destroy a result). By the Knaster-Tarski theorem, $T_N$ has a unique least fixpoint $F(N)$. Two reduction sequences may fire pairs in different orders, but each fires every pair at most once and adds the same results; both therefore converge to $F(N)$. □ □

**Corollary 0.7** (Independence of Evaluation Order): The fixpoint $F(N)$ is the same for every fair evaluation strategy (one that eventually fires every unresolved active pair). No scheduler achieves a strictly smaller fixpoint.

**Theorem 0.8** (Module-Layer Optimality): Under persistent firing, each distinct active pair fires at most once; no derivation step is repeated.

*Proof*: Once $a$ and $b$ are both resolved (after their pair fires), $(a, b)$ is never again an unresolved active pair. Any future active pair involving new result agents is distinct from $(a, b)$, not a re-firing of it. □ □

**Remark.** Theorem 2.4 establishes optimality in the Datalog semi-naive sense: no rule fires twice for the same instantiation. It does not establish Lévy-optimality (Lévy 1978) for the $\lambda$-calculus, which requires consuming semantics. Section 8 addresses this distinction for pure functional-logic evaluation.

**Remark (comparison with Δ-Nets perfect confluence).** Δ-Nets satisfies a stronger property — perfect confluence (Salvadori 2025, §3.3): any two normalizing orders reach the same result in the same number of interactions (a one-step diamond). PΔ-net

Monotone Confluence is weaker: it guarantees the same fixpoint but not the same interaction count across orders.

## 3. The Levels-Strata Theorem and the Translation $T$

### 3.1 Locally stratified Datalog programs

A **locally stratified Datalog program** $D$ consists of:

- an extensional database (EDB): a finite set of ground facts, at stratum 0;
- intensional database (IDB) relations $I = \{r_1, ..., r_k\}$;
- a stratum function $s : I \to \mathbb{N}$ with $s(h) \geq s(b)$ for every positive body literal $b$ and $s(h) > s(c)$ for every negated body literal $\neg c$ in a rule deriving $h$;
- rules $R$ of the form $h(\overline{x}) : -b_1(\overline{x}_1), ..., b_n(\overline{x}_n), \neg c_1(\overline{y}_1), ..., \neg c_m(\overline{y}_m)$.

Local stratification requires that the dependency graph, with positive and negative edges between relations, be acyclic across negation (Przymusinski 1988). Positive cycles within a stratum are permitted; negative cycles are not. Programs with negative cycles are treated in §4–§5.

### 3.2 The translation $T$

**Definition 0.9**: (Translation $T$). Given a locally stratified Datalog program $D$, the P$\Delta$-net $T(D)$ is constructed as follows.

**(T-EDB)** For each ground EDB fact $a(c_1, ..., c_k)$ at stratum 0: create an **EDB agent** — an eraser-shaped node with a named principal endpoint carrying the ground tuple as annotation (§3.5 fixes the canonical encoding). Level = 0.

**(T-IDB)** For each IDB relation $r_i$: create a canonical **relation replicator** $\text{Rep}(r_i)$ with level $l = s(r_i)$ and one auxiliary port per occurrence of $r_i$ as a body literal across all rules in $R$.

**(T-Rule)** For each rule $\rho$ of the form $h(\overline{x}) : -b_1(\overline{x}_1), ..., b_n(\overline{x}_n), \neg c_1(\overline{y}_1), ..., \neg c_m(\overline{y}_m)$:

- create a **rule fan** $F(\rho)$ — an application fan at level $s(h)$;
- for each positive body literal $b_i$: wire an auxiliary port of $\text{Rep}(b_i)$ to auxiliary port $i$ of $F(\rho)$, with **level delta** $\delta(\rho, b_i) = s(h) - s(b_i) \geq 0$;
- for each negative literal $\neg c_j$: wire an auxiliary port of $F(\rho)$ to an **eraser** $E(\rho, c_j)$ whose principal is connected to an auxiliary port of $\text{Rep}(c_j)$;
- wire the principal port of $F(\rho)$ to the auxiliary port of $\text{Rep}(h)$ assigned to $\rho$'s head occurrence.

**(T-Ground)** For a ground instantiation $\rho[\theta]$: relation replicators become **ground replicators** carrying the specific tuple; each EDB fact grounds into the level-0 structure of §3.5.

### 3.3 The Levels-Strata correspondence

**Theorem 0.10** (Levels-Strata Correspondence): Let $D$ be a locally stratified Datalog program with stratum function $s$. For every IDB relation $r$, the replicator $\text{Rep}(r)$ in $T(D)$ has level $s(r)$. For every occurrence of $r$ as a positive body literal in a rule with head $h$, the level delta at the corresponding auxiliary port is $s(h) - s(r) \geq 0$.

*Proof*: By construction. (T-IDB) defines $\text{Rep}(r)$ with level $s(r)$. (T-Rule) defines the level delta at the auxiliary port connecting $\text{Rep}(b_i)$ to $F(\rho)$ as $s(h) - s(b_i)$. Local stratification gives $s(h) \geq s(b_i)$ for positive literals, so $\delta \geq 0$. □ □

**Corollary 0.11** (Stratum arithmetic = level arithmetic): In $T(D)$, the level delta of an auxiliary port connecting a body relation at stratum $k_1$ to a rule head at stratum $k_2$ is $k_2 - k_1$. The $\Delta$-Nets status-machine constraint $0 \leq l_B - l_A \leq d$ corresponds exactly to the Datalog stratification constraint that a body literal at stratum $l_A$ can only contribute to a head at stratum $l_B$ with $l_B \geq l_A$.

**Corollary 0.12** (Negation = erasure): Negated body literals $\neg c$ are represented by erasers rather than replicator auxiliary ports. A rule that requires the absence of $c$ erases $c$'s contribution rather than propagating it — structurally identical to the $\Delta$-Nets treatment of affine abstraction (Salvadori 2025, §3.2).

### 3.4 Injectivity of $T$ and the image class

**Definition 0.13**: (T-net). A P$\Delta$-net $N$ is a **T-net** if there exists a locally stratified Datalog program $D$ with $N \cong T(D)$, where $\cong$ denotes graph isomorphism preserving agent-type annotations, wiring, levels, and level deltas. The class of all T-nets is denoted $\mathrm{Im}(T)$. Two programs are **equal up to relabeling** if there is a bijection $\rho$ on constants, extended to relation names, carrying the EDB and rules of one onto those of the other.

**Theorem 0.14** (Injectivity of \$T\$): If $T(D_1) \cong T(D_2)$ then $D_1 = D_2$ up to relabeling.

*Proof*: Let $\varphi : T(D_1) \to T(D_2)$ be an isomorphism. We recover each component of $D$ from the net; since $\varphi$ preserves agent type, level, arity, port connectivity, and level deltas, the recovered components of $D_1$ and $D_2$ agree up to the relabeling induced by $\varphi$.

**Step 1 (EDB).** By (T-EDB), each EDB fact produces a level-0 eraser-shaped node carrying a ground-tuple annotation, with no principal-to-principal wire to another agent at construction time. These three properties — level 0, ground-tuple annotation, no initial active-pair partner — are isomorphism invariants, so $\varphi$ maps EDB agents bijectively to EDB agents and preserves their annotations up to the induced constant bijection $\rho_c$. Hence $\mathrm{EDB}(D_1) = \rho_c(\mathrm{EDB}(D_2))$.

**Step 2 (IDB relations and strata).** By (T-IDB), each relation $r_i$ produces $\mathrm{Rep}(r_i)$ with level $s(r_i)$ and arity equal to its number of body occurrences. Relation replicators are distinguished from rule fans by wiring role: a relation replicator's principal is wired to rule fans, whereas a rule fan's principal is wired to a relation replicator (its head). $\varphi$ preserves type, level, and arity, so it maps $\mathrm{Rep}(r_i)$ to an agent of identical level and arity. The stratum function is read directly as $s(r_i) = \mathrm{level}(\mathrm{Rep}(r_i))$. Relations sharing a (level, arity) pair are separated by their wiring topology (Step 3). Hence the IDB and stratum function agree up to the relation renaming $\rho_r$ induced by $\varphi$.

**Step 3 (rules).** By (T-Rule), each rule $\rho$ produces a rule fan $F(\rho)$ whose connection topology is a complete fingerprint. Define the rule signature $\mathrm{sig}(F) = (\mathrm{level}(F), [(\mathrm{type}(\mathrm{nbr}(i)), \mathrm{level}(\mathrm{nbr}(i)), \mathrm{delta}(i)) : i \text{ an aux port of } F])$, where $\mathrm{nbr}(i)$ is the agent across the wire from aux port $i$. Then $\mathrm{sig}(F)$ encodes $s(h) = \mathrm{level}(F)$; for each positive body literal, a replicator neighbor with level $s(b_i)$ and delta $s(h) - s(b_i)$; and for each negative literal, an eraser neighbor whose principal traces one wire further to $\mathrm{Rep}(c_j)$. Two rules are identical iff their fans have the same signature and their heads connect at the same auxiliary port of the head replicator. Since $\varphi$ preserves types, levels, connectivity, and deltas, it maps rule fans of $D_1$ to rule fans of $D_2$ with identical signatures. Hence $R(D_1) = R(D_2)$ up to $\rho_r, \rho_c$.

Steps 1–3 exhibit the relabeling $(\rho_c, \rho_r)$ witnessing $D_1 = D_2$ up to relabeling. □ □

**Lemma 0.15** (Level delta encodes stratum gap): For any rule $\rho$ with head $h$ and positive body literal $b_i$, the level delta $\delta(\rho, b_i) = s(h) - s(b_i) \geq 0$ uniquely encodes the stratum gap, and $s(b_i) = \mathrm{level}(F(\rho)) - \delta(\rho, b_i)$ is recoverable from the net.

*Proof*: $\mathrm{level}(F(\rho)) = s(h)$ is readable from the fan; $\delta(\rho, b_i)$ is annotated on port $i$; subtraction gives $s(b_i)$, which must equal the independently readable level of $\mathrm{Rep}(b_i)$. □ □

**Corollary 0.16** (Bijectivity onto $Im(T)$): $T$ is a bijection from locally stratified Datalog programs up to relabeling onto $\mathrm{Im}(T)$. Injectivity is Theorem 3.4; surjectivity is Definition 3.3.

**Structural characterization of $\mathfrak{I}(T)$.** A P$\Delta$-net $N$ is a T-net iff all of the following hold:

(C1) **Non-negative deltas.** Every level delta on every replicator auxiliary port satisfies $\delta \geq 0$.

(C2) **Ground level-0 EDB agents.** Finitely many level-0 eraser-shaped nodes carry ground-tuple annotations.

(C3) **Stratum-monotone wiring.** If a replicator at level $l$ is connected via an auxiliary port of delta $d$ to a rule fan at level $k$, then $k = l + d \geq l$. No wire descends in level from a rule fan to a replicator.

(C4) **No initial fan-out replicators.** In $T(D)$, all replicators are fan-in; fan-out replicators appear only during persistent firing.

(C5) **Finite structure.** $|A| < \infty$.

(C6) **Eraser-mediated negation.** Every negative-literal connection has the form rule fan $\to$ eraser $\to$ relation replicator; negation never routes directly from a fan auxiliary port to a replicator principal.

These conditions are necessary by construction. They are sufficient: from a net satisfying C1–C6 one reads off the EDB from C2, the IDB and strata from C3, the rules from C3 and C6, and local stratification from C1 and C4 (no negative cycles). The recovered $D$ satisfies $T(D) \cong N$.

### 3.5 Canonical ground-term encoding

The treatment of ground tuples in (T-EDB) is a design choice; we fix it canonically and prove it adequate.

**Definition 0.17**: (ground constant agent). For a constant $c$, the **ground constant agent** $\mathrm{GC}(c)$ is a level-0 replicator with zero auxiliary ports — a nullary, hence eraser-shaped, node — carrying the label $c$. Its sole port is its principal port.

**Definition 0.18**: (ground tuple encoding). For a tuple $(c_1, ..., c_k)$, the encoding $\mathrm{Chain}(c_1, ..., c_k)$ is the sequence $\mathrm{GC}(c_1) - \mathrm{GC}(c_2) - \cdots - \mathrm{GC}(c_k)$, each connected principal-to-principal to the next, with $\mathrm{GC}(c_k)$ connected to the EDB agent's principal port. For $k = 0$ (a propositional constant) no chain is needed; the EDB agent itself carries the relation name.

**Definition 0.19**: (EDB agent, canonical form). For each ground EDB fact $a(c_1, ..., c_k)$, the EDB agent is an eraser $E(a)$ labeled with the relation name $a$ and level 0, whose principal is connected to the tail of $\mathrm{Chain}(c_1, ..., c_k)$. All agents in the structure are at level 0.

**Theorem 0.20** (Adequacy of the ground-term encoding): The encoding of Definitions 3.5–3.7 supports (a) ground-fact creation, (b) tuple matching via annihilation, and (c) distinguishability.

*Proof*: **(a) Creation.** For each EDB fact $a(c_1, ..., c_k)$, the chain together with $E(a)$ is a closed finite level-0 subgraph present in $T(D)$ by Definition 3.7. No rule is needed: the EDB agent carries no incoming principal wire from an unresolved agent, so it is resolved at initialization.

**(b) Matching.** Ground tuples match iff equal componentwise. In the replicator×replicator semantics of $\Delta$-Nets (Salvadori 2025, §4), two replicators annihilate iff they share the same level and are equal (same label, arity, deltas). $\mathrm{GC}(c)$ and $\mathrm{GC}(c')$ are both level-0, arity-0, delta-free, so they annihilate iff $c = c'$ — exactly ground unification. The chain structure enforces positional matching: $\mathrm{GC}(c_i)$ aligns with $\mathrm{GC}(c'_i)$ at position $i$, and all $k$ positions must annihilate for the tuple to unify.

**(c) Distinguishability.** Distinct tuples yield non-isomorphic chains. If lengths differ ($k \neq k'$) the chains have different agent counts. If lengths agree but $c_i \neq c'_i$ at some $i$, the label sequences differ at position $i$; since labels are isomorphism invariants, no isomorphism maps one chain to the other. □ □

**Corollary 0.21** (Injectivity with structural tuples): With the canonical encoding, Theorem 3.4 holds without treating ground-tuple annotations as atomic labels: an isomorphism maps level-0 GC agents to level-0 GC agents preserving chain structure and labels, inducing the same constant relabeling $\rho_c$ as in Step 1.

The encoding is minimal (reuses existing agent types and level 0), adequate (Theorem 3.5), canonical (chain order matches argument order), and compatible with stratification (all ground agents at level 0, below every IDB stratum). Alternative encodings (binary trees, or a single $k$-ary agent) would also satisfy adequacy; we fix the chain form to make $T$ deterministic.

## 4. The Status Machine as Well-Founded Semantics

### 4.1 WFS three-valued logic

The well-founded semantics (Van Gelder, Ross & Schlipf 1991) assigns each ground atom $a$ of a program one of three values:

- **True** ($a \in M^+$): $a$ is definitively derivable;
- **False** ($a \in M^-$): $a$ is definitively not derivable under the closed-world assumption;
- **Undefined** ($a \in M^\perp$): $a$ lies in a negative cycle; its value cannot be determined.

For locally stratified programs, $M^\perp = \varnothing$. For arbitrary programs with negative cycles, WFS assigns undefined to the contested atoms.

### 4.2 The status machine and the three net classes

We recall the Δ-Nets status machine (Salvadori 2025, §5.3): replicators start unpaired; interacting with a fan makes both results unknown; an unknown replicator consecutive to an unpaired one with $0 \le l_B - l_A \le d$ is determined unpaired, because no replicator can interact with $B$ before $A$ annihilates, and an unpaired $A$ never annihilates. The status machine thus detects replicators that will **never** find a paired fan-out to annihilate with — blocked by the topology, not by a runtime check.

**Definition 0.22**: (Status-value correspondence). Fix a program $D$ (with negation permitted) and its ground dependency graph $G_D$ on the finite Herbrand base $B_D$, with a positive edge $h \xrightarrow{+} p_i$ and a negative edge $h \xrightarrow{-} q_j$ for each ground rule instance $h \leftarrow p_1, ..., p_n, \neg q_1, ..., \neg q_m$. At the fixpoint $F(T(D))$ every atom $a \in B_D$ is in exactly one state:

- **resolved** ($a \in$ Res): Rep($a$) exists and fired (via annihilation or rule-fan firing) → assigned value **true**;
- **unpaired** ($a \in$ Unp): Rep($a$) exists but never fired → assigned value **undefined**;
- **absent** ($a \in$ Abs): no agent for $a$ was generated → assigned value **false** (closed-world).

A cycle of $G_D$ is **negative** if it traverses an odd number of negative edges; $D$ is locally stratified iff $G_D$ has no negative cycle (Przymusinski 1988).

### 4.3 The operational contract

To reason about fixpoint status we fix the minimal operational semantics forced by $T$ (§3.2) and the persistent firing rule (§2.2). Each axiom is justified by a construction clause, not freely assumed.

- **(A1) Demand-generation.** Rep($a$) is generated iff $a$ is the head of some ground rule instance, or $a \in$ EDB. An atom with no defining instance and not in the EDB is never generated — it is absent.
- **(A2) EDB immediacy.** An EDB agent is resolved at once (§3.5, Theorem 3.5(a)). So EDB ⊆ Res.
- **(A3) Rule-fan firing.** A rule fan for $h \leftarrow p_1, ..., p_n, \neg q_1, ..., \neg q_m$ fires — resolving Rep($h$) — iff every positive premise replicator Rep($p_i$) has fired and every negative eraser $E(\rho, q_j)$ has fired.
- **(A4) Eraser-firing as blocking.** The eraser for $\neg q_j$ fires at the moment stratum $s(q_j)$ of the net has reached its own fixpoint and Rep$(q_j)$ is not resolved there ($q_j \in$ Abs ∪ Unp); it does not fire while $q_j \in$ Res.

The guiding symmetry: a positive premise helps a rule fire by *being resolved*; a negative premise helps by *failing to be resolved*.

**Definition 0.23**: (blocking relation). Atom $a$ is **blocked by** $c$, written $a \lhd c$, when in every defining rule instance $\rho$ of $a$ the fan fails to fire and $c$ is a premise supplying the obstruction: either $c = p_i$ is a non-resolved positive premise, or $c = q_j$ is a resolved negative premise whose eraser fired.

### 4.4 The biconditional

Proposition 4.2 below is the "negative cycle ⇒ unpaired" direction; Theorem 4.3 is the converse. Together they give the biconditional that makes the status–WFS correspondence total.

**Proposition 0.24** (Negative cycle $=>$ unpaired): If atoms in a negative cycle of $G_D$ are not resolved by support from outside the cycle, their replicators are unpaired at $F(T(D))$.

*Proof*: Consider a minimal negative cycle; take the simplest case $a$ depends on $\neg b$ and $b$ depends on $\neg a$. By $T$, the dependency of $a$ on $\neg b$ is an eraser $E(a, b)$ between $a$'s rule fan and $\text{Rep}(b)$. By (A3), $\text{Rep}(a)$ resolves only if $a$'s rule fan fires, which by (A4) requires $E(a, b)$ to fire, i.e. $b \notin \text{Res}$. Symmetrically $\text{Rep}(b)$ resolves only if $a \notin \text{Res}$. If neither has outside support, each can fire only once the other has failed to fire, and neither ever does: both remain unfired. At fixpoint both are unpaired. The argument lifts to any negative cycle by iterating the mutual block along the odd-negative-edge loop. □ □

**Theorem 0.25** (Unpaired $=>$ negative cycle): If $a \in \text{Unp}$ then $a$ lies on a negative cycle of $G_D$.

*Proof*: We argue through four steps. Throughout, status is monotone (Theorems 2.1, 2.3): a resolved fan stays resolved, and a fan whose premises are settled and which has not fired stays unfired.

**Step 1 ($a$ is blocked).** Since $a \in \text{Unp}$, $\text{Rep}(a)$ was generated, so by (A1) $a$ is a head of some instances, or $a \in \text{EDB}$; the latter is excluded by (A2) since $\text{EDB} \subseteq \text{Res}$. Thus $a$ heads $k \geq 1$ instances, none of whose fans fired. Fix any such instance $a \leftarrow p_1, ..., p_n, \neg q_1, ..., \neg q_m$. By (A3) its failure to fire means, permanently, either some positive $p_i$ is not resolved ($p_i \in \text{Unp} \cup \text{Abs}$) or some eraser $E(\rho, q_j)$ did not fire, which by (A4) means $q_j \in \text{Res}$. So $a \lhd c$ for some $c$.

**Step 2 (absence never suspends).** Suppose $a \lhd p$ via a positive premise $p$ with $p \in \text{Abs}$. By (A1), $p \in \text{Abs}$ means $p$ has no defining instance and $p \notin \text{EDB}$ — for if $p$ had a defining instance its replicator would have been generated, contradicting $p \in \text{Abs}$. Then $p$ is determinately (closed-world) false: a *settled* obstruction, not a suspended one. A defining instance of $a$ whose only obstruction is a determinately-false positive premise is a *dead* instance — permanently and determinately unfireable. If every defining instance of $a$ is dead, then by the three-valued consequence reading $a$ is settled false, i.e. $a \in \text{Abs}$ — contradicting $a \in \text{Unp}$. Hence for $a$ to remain undefined it must retain at least one *live, suspended* defining instance, whose obstruction is a positive premise in Unp or a negative premise in Res whose own truth is not settled. Absence is a sink that kills a rule instance; it is never a link that suspends a head.

**Step 3 (finiteness forces a cycle).** Build the **suspension graph** $H$ on the undefined atoms (plus resolved atoms occurring as live negative blockers), with edge $a \to c$ whenever $a \lhd c$ through a live, suspended obstruction: a positive edge $a \xrightarrow{+} p$ when $p \in \text{Unp}$ is a non-resolved positive premise of a live instance of $a$; a negative edge $a \xrightarrow{-} q$ when $q \in \text{Res}$ is a resolved negative premise whose eraser fired. By Steps 1–2 every $a \in \text{Unp}$ has at least one out-edge. Every edge of $H$ is an edge of $G_D$ by construction. Starting from $a$ and walking out-edges, a positive edge stays in Unp; a negative edge reaches $q \in \text{Res}$. If $q$ were *firmly true* — resolved through premises grounded entirely in settled atoms — then $\neg q$ would be a determinately-false, settled obstruction of $a$, killing that instance, contradicting that $a$ retains a live suspended instance. Hence the suspending $q$ is *not* firmly true: its own resolution depends, through its satisfied body, on an atom in Unp, giving a $G_D$-path from $q$ back into Unp. The walk therefore re-enters Unp after finitely many steps. In the finite graph $G_D$ a walk that re-enters Unp infinitely often repeats a vertex, exhibiting a cycle $C$ through $a$.

**Step 4 (the cycle is negative).** Suppose $C$ is positive (an even number — possibly zero — of negative edges). If zero negative edges, $C$ is a pure positive recursion; by Theorem 2.3 the accumulation operator is monotone and reaches its least fixpoint. A pure positive cycle has no negation to create suspension: either some member has an alternative live instance grounded outside $C$ and so is resolved (contradicting Unp), or no member is independently supported and the least fixpoint resolves none of them, making each a determinately-false unsupported atom, i.e. in Abs (contradicting Unp). If the number of negative edges is even and nonzero, assign a parity $\pi$ along $C$ flipping on each negative edge; evenness makes the parity consistent on closing the loop, so a definite two-valued valuation propagates with no contradiction. Formally the lower and upper Van

Gelder iterates meet on $C$, making every atom of $C$ determinate (in Res $\cup$ Abs) — again contradicting Unp. Only an **odd** number of negative edges makes the parity inconsistent, so the two iterates fail to meet and the atoms stay undefined. Hence $C$ is a negative cycle.

Steps 1–4 place $a$ on a negative cycle of $G_D$. □□

**Corollary 0.26** (Status–WFS biconditional): In $F(T(D))$, $a \in$ Unp iff $a$ lies on a negative cycle of $G_D$ not resolved by outside support. ($\Rightarrow$ Theorem 4.3; $\Leftarrow$ Proposition 4.2.) Consequently, for locally stratified $D$ the graph $G_D$ has no negative cycle, so Unp $= \varnothing$ and the status–WFS correspondence is total.

## 5. Fixpoint = WFS Model

We now prove that the P$\Delta$-net fixpoint equals the well-founded model for **all** programs, via Van Gelder's alternating fixpoint.

### 5.1 The alternating fixpoint

For a set $I \subseteq B_D$ of assumed-false atoms, define $T_{\{D,I\}}(X)$ to fire $h$ whenever some instance $h \leftarrow p_1, ..., p_n, \neg q_1, ..., \neg q_m$ has $\{p_1, ..., p_n\} \subseteq X$ and $\{q_1, ..., q_m\} \subseteq I$. Let $A_D(I) = \mathrm{lfp}\big(T_{\{D,I\}}\big)$. The operator $A_D$ is **antimonotone** in $I$ (a larger false-set licenses more firings), so $A_D \circ A_D$ is monotone and, by Knaster-Tarski, has a least fixpoint $K$ and greatest fixpoint $U$ with $K \subseteq U$. The **well-founded model** is $M^+ = K$ (true), $M^- = B_D \setminus U$ (false), $M^\perp = U \setminus K$ (undefined). Equivalently the alternating sequence $J_k = (A_D \circ A_D)^k(\varnothing)$ increases to $K$ and $J'_k = (A_D \circ A_D)^k(B_D)$ decreases to $U$; an atom is undefined iff it stays in the upper estimate but never enters the lower one.

### 5.2 Net iterates mirror the alternating sequence

**Lemma 0.27** (Net lower/upper iterates): Define, from the net evaluation of $T(D)$ under (A1)–(A4):

- $L_k$ = atoms resolved by firing using only premises established within the first $k$ negation rounds;
- $U_k$ = atoms not determinately absent after $k$ negation rounds (generated and not yet refuted).

Then $L_k = J_k$ and $U_k = J'_k$ for all $k$; the net's round-by-round lower/upper estimates coincide with Van Gelder's alternating iterates.

*Proof*: By induction on $k$. At $k = 0$, with no negation round performed, the only unconditionally resolvable atoms are the EDB facts (A2) and heads of negation-free instances whose positive premises are transitively EDB — which is lfp of the positive part with every negative premise satisfiable, matching the base $J_0$. Dually $U_0 = J'_0$.

For the step, assume $L_k = J_k$ and $U_k = J'_k$. A negation round at a state with lower estimate $L_k$ and upper estimate $U_k$ does exactly this by (A3)+(A4): an eraser $E(\rho, q)$ fires ($\neg q$ deemed satisfied) iff $q \notin U_k$ — matching "assume $q$ false when $q$ is outside the upper estimate"; a rule fan then fires $h$ iff all positive premises are in $L_k$ and all erased negatives are satisfied — matching $T_{\{D,\cdot\}}$ firing $h$ when positives are in the lower estimate and negatives outside the upper estimate. This is one application of $A_D$ composed to restore the lower estimate, i.e. one step of $A_D \circ A_D$. Hence $L_{\{k+1\}} = (A_D \circ A_D)(L_k) = J_{\{k+1\}}$ and $U_{\{k+1\}} = J'_{\{k+1\}}$. □ □

**Termination.** $B_D$ is finite, $J_k$ increases and $J'_k$ decreases in a finite lattice, so both stabilize after finitely many rounds at $K$ and $U$. By Monotone Confluence (Theorem 2.3) the net reaches its fixpoint independent of scheduling; the above shows that fixpoint carries the data $(K, U)$.

### 5.3 The main theorem

**Theorem 0.28** (Fixpoint = WFS, all programs): For every Datalog program $D$ with negation, the P$\Delta$-net fixpoint $F(T(D))$ satisfies

- $a \in$ Res $\Leftrightarrow a \in M^+$ (true),
- $a \in$ Unp $\Leftrightarrow a \in M^\perp$ (undefined),
- $a \in$ Abs $\Leftrightarrow a \in M^-$ (false),

where $M = (M^+, M^\perp, M^-)$ is the well-founded model of $D$. The former Conjecture holds for all programs, not only locally stratified ones.

*Proof*: By Lemma 5.1 the stabilized lower estimate is $K$ and the upper estimate is $U$.

**(Resolved = true.)** $a \in$ Res iff some fan or annihilation for $a$ fired at the stabilized state, iff $a \in \bigcup_k L_k = K = M^+$. Conversely $a \in K$ means $a$ enters the increasing lower estimate at some round, which by Lemma 5.1 is the round at which a defining fan of $a$ fires, so $a \in$ Res.

**(Absent = false.)** $a \in$ Abs iff $a$ is never generated (A1) or every defining instance is determinately killed (Theorem 4.3, Step 2); in alternating-fixpoint terms $a$ never enters any lower

estimate and is expelled from the upper estimate, i.e. $a \notin U$, i.e. $a \in B_D \setminus U = M^-$. Conversely $a \in M^-$ means $a$ is dropped from the decreasing upper estimate, which by Lemma 5.1 is the round at which every defining instance of $a$ is refuted, so no agent for $a$ survives — $a \in$ Abs.

**(Unpaired = undefined.)** $a \in$ Unp iff Rep($a$) is generated but never fires, iff (Lemma 5.1) $a$ is in $U$ (never refuted, so an agent persists) but never in $K$ (never fires), iff $a \in U \setminus K = M^{\perp}$. As a cross-check with §4: $a \in$ Unp $\Rightarrow$ (Theorem 4.3) $a$ lies on a negative cycle, and the hallmark of $M^{\perp} = U \setminus K$ is exactly that $a$ oscillates between the alternating iterates. □ □

**Corollary 0.29** (Soundness): The net never over-derives (no $M^-$ atom is resolved) nor under-derives (no $M^+$ atom is absent).

**Corollary 0.30** (Completeness on the locally stratified fragment): If $D$ is locally stratified then $G_D$ has no negative cycle, so by Theorem 4.3 Unp $= \varnothing$, hence $M^{\perp} = \varnothing$ and the net computes the two-valued perfect model. The general Theorem 5.1 strictly subsumes the stratified case.

**Corollary 0.31** (Scheduling independence of the three-valued answer): By Monotone Confluence the fixpoint $F(T(D))$ is unique and order-independent; Theorem 5.1 shows the induced reading (Res, Unp, Abs) equals the unique well-founded model. The only ordering constraint required is the negation-round discipline of (A4): negative premises are read only after the premise's own round settles.

## 6. Phase Structure and Semi-Naive Evaluation

### 6.1 Two phases in Δ-Nets

Δ-Nets for $\Delta I/\Delta K$ reduces in two phases (Salvadori 2025, §5.4). **Phase 1** applies the core interaction rules and unpaired-replicator merging in leftmost-outermost order until no further reduction applies. **Phase 2** replaces the core fan-replication rule with the auxiliary fan-replication rule, distributing lower-level replicators to higher-level contexts to produce the canonical net.

### 6.2 Phase 1 = semi-naive intra-stratum fixpoint

Fix a single stratum index $k$ and the stratum-$k$ rule set $R_k$. Let $\mathrm{EDB}_k$ be the atoms at strata $< k$ already derived (true in WFS by induction on strata). Define the **immediate-consequence operator** $T_k(I) = I \cup \{h(\overline{a}) :$ some $\rho \in R_k$ and grounding $\theta$ yields head $h(\overline{a})$, all positive $k$ are in $\mathrm{EDB}_k$, and all negated literals hold under the CWA w.r.t. $T_k$ is monotone and inflationary over the finite lattice of ground stratum-$k$ atoms, so $\mathrm{lfp}(T_k)$ exists and is reached from $\mathrm{EDB}_k$. **Semi-naive evaluation** computes $\mathrm{lfp}(T_k)$ by iterating Δ-rounds that fire only instances using at least one newly derived stratum-$k$ body literal. **Phase 1** is the restriction of persistent firing to (a) same-level (level-$k$) replicator-fan pairs and (b) unpaired merging of consecutive zero-delta level-$k$ replicators; it runs until no same-level active pair remains. Let $\Phi_k$ be the set of ground atoms whose replicator is resolved at Phase-1 fixpoint.

**Theorem 0.32** (Phase 1 = semi-naive fixpoint): $\Phi_k = \mathrm{lfp}(T_k)$.

*Proof*: **($\Phi_k \subseteq \mathrm{lfp}(T_k)$.)** By induction on the resolution-chain depth of Rep($r[\theta]$). Base: a fan $F(\rho[\theta])$ fires whose positive body literals are all at strata $< k$, hence in $\mathrm{EDB}_k$; by (A3) $F$ fires only when all body replicators are resolved, so $r(\overline{\theta}) \in T_k(\mathrm{EDB}_k) \subseteq \mathrm{lfp}(T_k)$. Step: $F(\rho[\theta])$ fires with at least one stratum-$k$ positive body literal $b_j$; by (A3) each stratum-$k$ $\mathrm{Rep}(b_j[\theta_j])$ is resolved, hence by inductive hypothesis $b_j \in \mathrm{lfp}(T_k)$, while strata-$< k$ literals are in $\mathrm{EDB}_k$ and all negated relations settled before Phase 1 (A4); so all body conditions of $\rho[\theta]$ hold in $\mathrm{lfp}(T_k)$ and $r(\overline{\theta}) \in \mathrm{lfp}(T_k)$.

**($\mathrm{lfp}(T_k) \subseteq \Phi_k$.)** $\mathrm{lfp}(T_k) = \bigcup_i T_k^i(\mathrm{EDB}_k)$; induct on $i$. At $i = 0$, stratum-$k$ EDB atoms are level-$k$ EDB agents, resolved by construction (§3.5). At $i \to i+1$, if $r(\overline{a}) \in T_k^{i+1} \setminus T_k^i$ via rule $\rho$ and grounding $\theta$, then all positive stratum-$k$ premises are in $T_k^i$ (resolved by inductive hypothesis) and all strata-$< k$ premises in $\mathrm{EDB}_k$ (resolved before Phase 1). By persistent firing, once all body replicators of $F(\rho[\theta])$ are resolved they form active pairs with $F$; all agents are at level $k$, so the same-level condition holds and the pair fires, resolving Rep($r[\theta]$).

**(Termination.)** By Theorem 2.4 each distinct active pair fires at most once; the ground stratum-$k$ base is finite (function-free Datalog, finite EDB); each step adds at least one resolved

replicator; hence Phase 1 terminates in at most |ground stratum-k base| steps, at which point $\Phi_k = \text{lfp}(T_k)$. □ □

**Remark (intermediate states).** Phase-1 order is free (Theorem 2.3), while semi-naive proceeds in discrete rounds. The two sequences of intermediate states need not coincide. Only the final fixpoints are required to agree (Theorem 6.1), and they do. Order-independence is a feature (Full Parallelism, Theorem 2.2), not a deficiency.

### 6.3 Phase 2 = cross-stratum materialization

Phase 2 fires when a replicator's principal port connects to a fan's auxiliary port, distributing lower-level content to higher-level contexts. In PΔ-net terms: once stratum $k$ has reached fixpoint (Phase 1), its derived facts are materialized as inputs to rules at strata $k+1, k+2, \ldots$. The auxiliary-fan replication effect — a lower-level replicator distributing itself through a fan to higher-level contexts — is exactly stratum-$k$ fact injection into stratum-$(k+\delta)$ rule evaluation.

## 7. CUE Semantics as External Validation

### 7.1 CUE's semantic model

CUE (Hăvărneanu 2026) is a language for data and schema unification. Its model assigns each description $D$ a subset $[[D]]$ of complete data satisfying $D$; refinement is subset inclusion, $D_1 \leq D_2$ iff $[[D_1]] \subseteq [[D_2]]$, and unification is intersection, $[[D_1 \wedge D_2]] = [[D_1]] \cap [[D_2]]$. The fundamental monotonicity property (Hăvărneanu 2026, Theorem 2.7): for any composite $Q$ of reindexing and projection, $Q(P') \subseteq Q(P)$ whenever $P' \subseteq P$ — additional constraints only restrict; new information never invalidates old.

> **Proposition 0.33** (Hăvărneanu 2026, Prop. 4.2 — Finite-Basis Saturation): On a fixed finite fact-and-obligation basis, fair iteration of finitely many monotone inflationary propagators reaches the least common fixed point above the initial information; this fixed point is independent of scheduling.

This is a specialization of Knaster-Tarski to CUE's propagator lattice, and is structurally identical to P Δ-net Theorem 2.3 (Monotone Confluence), derived independently by a different route.

### 7.2 CUE as a semantic instance of PΔ-nets

| CUE concept | PΔ-net structure |
|---|---|
| Description $D$ as predicate $[[D]]$ | Agent type annotation |
| Live reference (unresolved field) | Unresolved agent |
| Completed field (resolved description) | Resolved agent |
| Unification $D_1 \wedge D_2$ | Fan-annihilation (conjunction of two body conditions) |
| $\bot$ (empty predicate) | See §7.3: encodable as a distinguished IDB relation |
| Monotone refinement (Thm 2.7) | Monotone accumulation (Theorem 2.1) |
| Finite-basis saturation (Prop. 4.2) | Monotone Confluence (Theorem 2.3) |
| Exact residualization (eq. 17) | Progenitor retention: resolved agents retain wire dependencies |

Table 1: CUE–PΔ-net correspondence

CUE's domain (descriptions as constraints on attribute records) is precisely the domain of Nix attribute sets; CUE's monotone refinement is what Nix module composition does — merging attribute sets only adds constraints, never contradicting prior fields in the pure, non-override case. CUE's Corollary 5.11 (Monotone Certified Refinement) is, in PΔ-net terms, the monotonicity guarantee (Theorem 2.1): once an agent is resolved, further derivation can only add resolved agents, never un-resolve one.

### 7.3 Failure propagation: scoped out

CUE has an explicit failure $\bot$ that propagates through conjunctions (`int & string` $= \bot$). We show this requires **no** extension of PΔ-nets.

**False ≠ contradiction.** Under WFS, false means no derivation path reaches $a$; CUE's $\bot$ means two constraints jointly denote the empty set — a contradiction. These are distinct. WFS is a coherent three-valued model over a fixed signature; it produces true/false/undefined and never a contradiction. There is therefore no fourth value to model inside the monotone PΔ-net layer.

**Constraint violation is an IDB derivation.** When integrity violations must propagate, introduce a distinguished relation `bot` at the maximum stratum and encode:

```
constraint_violated :- body(Cᵢ)   for each
constraint Cᵢ
```

```
bot                 :- constraint_violated
output(X)           :- derived(X), not bot
```

**Theorem 0.34** (Encodability of $bot$-propagation): For a finite set of integrity constraints $C$ over a program $D$, under the above encoding `bot` is WFS-true at fixpoint iff some $C_i$ is violated, and `output(X)` is WFS-false for all $X$ iff `bot` is true; the propagation behavior is identical to CUE's $\perp$ -propagation through conjunction.

*Proof*: By the WFS fixpoint: `constraint_violated` is derived iff some $C_i$ body is satisfied; `bot` is derived iff `constraint_violated` is; and by the negated literal `not bot`, `output(X)` is WFS-false iff `bot` is WFS-true. Each step is a standard WFS derivation requiring no new agent type. □ □

**Corollary 0.35**: PΔ-nets require no $\perp$ agent type to handle failure propagation for any program with integrity constraints; CUE's $\perp$ is a semantic instance of IDB derivation of a distinguished failure relation. Native type-error propagation inside expression bodies belongs to the consuming Δ-Nets expression layer (§8), not to the monotone module layer.

## 8. Optimal Evaluation of Pure Functional-Logic Languages

### 8.1 Two layers of evaluation

A pure functional-logic language combines two concerns:

1. the **logic/module layer** — deriving facts from rules over a database, possibly recursively and mutually (the Datalog layer), with monotone, set-valued semantics;
2. the **expression/body layer** — evaluating pure functional expressions ($\lambda$-terms, arithmetic, constructors) in rule bodies and heads, with consuming, value-valued semantics ($\beta$-reduction destroys the redex).

Optimal evaluation matches the machine model to the layer:

- **logic-layer optimality** (Datalog semi-naive): each ground fact derived at most once;
- **expression-layer optimality** (Lévy 1978): each family of $\beta$-reductions, defined by sharing structure, fires exactly once.

### 8.2 Pure-Nix module system as locally stratified Datalog

Consider the pure fragment of Nix (no side effects). Attribute sets `{ k = e; ... }` are ground facts attr(module, key, value); recursive attribute sets `rec { ... }` are same-stratum IDB rules; `import` is rule-base union; `lib.fix` is explicit fixpoint over a recursive IDB relation; `lib.optionalAttrs` / `if ... then ... else {}` are conditional rules; and negation-as-absence (`builtins.tryEval`, `lib.hasAttr`) is a WFS three-valued query.

**Proposition 0.36** (Pure-Nix is locally stratified Datalog): The pure-Nix module system, restricted to finite modules with acyclic import graphs and no circular negated attribute references, encodes as a locally stratified Datalog program under $T$.

*Proof*: By structural induction on Nix module syntax. Attribute definitions with no cross-references encode as EDB facts; recursive definitions within a `rec` block as same-stratum IDB rules; `import` as rules added at stratum 0 (imported EDB) or the importer's stratum (imported IDB); conditional derivation as a rule with a condition body literal; negated conditions as negated literals. Acyclic imports and the absence of circular negation guarantee local stratification. □ □

### 8.3 Module-layer optimality via PΔ-nets

By Theorem 2.4 and Proposition 8.1, a PΔ-net evaluation of $T$(pure-Nix module $M$) derives each attribute value at most once — the semi-naive optimality guarantee. By Theorem 2.2 all independent attribute derivations proceed in parallel. By Theorem 2.3 the fixpoint $F(T(M))$ is unique and order-independent. By Theorem 5.1 that fixpoint is the well-founded model of the module system.

### 8.4 Expression-layer optimality requires consuming Δ-Nets

**Proposition 0.37** (Expression bodies require consuming $Delta$-Nets): For general $\lambda$-term expression bodies with sharing, PΔ-nets do not achieve Lévy-optimality; consuming Δ-Nets are necessary.

*Proof*: Lévy-optimality requires that families of $\beta$ -reductions sharing a common ancestor redex fire exactly once. In Δ-Nets this is achieved by merging unpaired replicators — two copies of a shared,

not-yet-demanded subterm are merged before they expand (Salvadori 2025, §5.3). In PΔ-nets a resolved replicator cannot merge with another, since merging would be a re-firing (Theorem 2.4). Hence two copies of the same shared subterm, if separately demanded by two consumers, may each be independently reduced, duplicating work. □□

### 8.5 Hybrid architecture

**Theorem 0.38** (Hybrid optimality): A hybrid system in which the module layer is evaluated by PΔ-nets and expression bodies within rule heads by consuming Δ-Nets achieves: module-layer optimality (each ground attribute derived at most once); expression-layer Lévy-optimality (each $\beta$-reduction family fires once); monotone confluence of the module fixpoint; perfect confluence of each expression body to a unique normal form; and full parallelism at both layers.

*Proof*: Module-layer optimality is Theorem 2.4 on the PΔ-net layer. Expression-layer Lévy-optimality is Salvadori (2025, Theorem 7.1) on consuming Δ-Nets per rule body. Monotone confluence is Theorem 2.3; perfect confluence of expression bodies is Salvadori (2025, §3.3). Module-layer parallelism is Theorem 2.2; expression-layer parallelism follows from perfect confluence. The layers communicate via a one-way interface: each expression normal form (consuming Δ-Nets output) yields exactly one ground agent (PΔ-net input), and ground agents accumulate monotonically. Hence the composition preserves all five properties. □ □

**Theorem 0.39** (Main result): A PΔ-net system equipped with consuming Δ-Nets for expression bodies evaluates pure-Nix so that each module-level attribute value is derived at most once, each expression body is reduced Lévy-optimally, evaluation is confluent at both layers, independent subcomputations proceed in parallel, and the result is the well-founded model of the pure-Nix module system. A PΔ-net system alone achieves all of this except expression-layer Lévy-optimality: it evaluates pure Nix correctly with module-layer optimality but may redundantly re-evaluate $\lambda$-terms shared across rule instantiations.

*Proof*: Immediate from Theorem 8.3 (hybrid) and Proposition 8.2 (the PΔ-net-only gap), with correctness of the module-layer result by Theorem 5.1 and Proposition 8.1. □ □

### 8.6 Incremental maintenance under retraction: a layering

PΔ-nets are monotone (Theorem 2.1): un-resolving an agent is not a PΔ-net operation. Retraction of an EDB fact $f$ is handled at a layer above the monotone fixpoint.

**Theorem 0.40** (DRed composes with P$Delta$-nets): Let $N_0 = T(D, E)$ with fixpoint $F(N_0) = M(D, E)$. Given a retraction $E' = E \setminus \{f\}$, the composite that (i) computes each resolved agent's provenance, (ii) deletes the affected set $\mathrm{Aff}(f) = \{a \text{ resolved} : f \in \mathrm{Prov}(a)\}$, and (iii) re-runs persistent firing from $N_1 = T(D, E')$ to fixpoint, computes $M(D, E')$ correctly.

*Proof*: Within the rederivation phase (iii) the PΔ-net evaluation is purely monotone — facts only accumulate, no retraction occurs inside $T(D, E')$ — so by Theorem 2.3 $F(N_1)$ is uniquely determined, and by Theorem 5.1 $F(N_1) = M(D, E')$. The delete phase (ii) is a meta-level operation on the fixpoint representation, outside the PΔ-net firing rule; monotonicity within each individual computation is preserved. DRed is correct for stratified Datalog with negation (Gupta, Mumick & Subrahmanian 1993, Theorem 3.1); since PΔ-nets compute $M(D, E)$ correctly (Theorem 5.1), the composition computes $M(D, E')$ correctly. □ □

**Provenance** is latent in the wire graph: define $a \prec b$ iff $b$'s wires contributed to $a$'s firing; then $\mathrm{Prov}(a) = \{e \in \mathrm{EDB} : e \prec a\}$, computable by backward traversal from $a$ to EDB leaves, bounded by $a$'s stratum (Theorem 3.1). A differential-dataflow layer (McSherry, Murray, Isaacs & Isard 2013) is an equally valid alternative. In both cases no modification of the PΔ-net calculus is required; retraction is a property of the enclosing layer.

## 9. Structural Correspondence Table

| **Δ-Nets concept** | **PΔ-nets / Datalog interpretation** |
|---|---|
| Agent | Ground atom / derivable fact |
| Fan (@ application) | Rule body join (conjunction of two conditions) |
| Fan ($\lambda$ abstraction) | Derived head fact (result of a rule firing) |
| Replicator (level $l$, deltas $d_0 ... d_n$) | IDB relation at stratum $l$, used $n+1$ times as body literal |
| Eraser | Negated literal ($\neg c$: $c$'s contribution probed, not propagated) |
| Level $l$ of a replicator | Stratum $s(r)$ of IDB relation $r$ |
| Level delta $d_i$ at aux port $i$ | Stratum distance $s(\text{head}) - s(\text{body literal } i) \geq 0$ |
| Ground constant agent GC($c$) | Constant symbol $c$ (level-0 nullary) |
| Chain of GC agents | Ground tuple (positional) |
| Active pair fires (consuming) | Rule application: destructive substitution |
| Active pair fires (persistent) | Rule firing: head derived, body retained |
| Fan×fan annihilation ($\beta$) | Rule fires: head produced from two joined body facts |
| Rep×rep annihilation (same level) | Same-stratum recursive derivation: two instances agree and combine |
| Rep×rep commutation ($l < k$) | Cross-stratum dependency: lower-stratum fact used by higher-stratum rule |
| Rep×eraser | Negative literal check: negated relation probed, not consumed |
| Unpaired replicator at fixpoint | WFS-undefined atom (on an unresolved negative cycle) |
| Unknown status replicator | WFS atom with pending evaluation |
| Resolved (annihilated) replicator | WFS-true atom (derivation completed) |
| Absent agent at fixpoint | WFS-false atom (closed-world) |
| Status constraint $0 \leq$ [illegible] | Stratification: body stratum $\leq$ head stratum, gap bounded by level delta |

Table 1. Δ-Nets and PΔ-nets / Datalog

## 10. Future Work

The soundness questions of the calculus are settled by §2–§5. The remaining open directions are genuine research and engineering, not gaps in the theory established here.

1. **DRed design specifics.** Theorem 8.5 establishes that DRed composes correctly with the monotone PΔ-net layer and that provenance is latent in the wire graph. The concrete engineering of provenance tracking — compact provenance representations, incremental affected-set computation, and the delete/rederive scheduling that minimizes recomputation — remains to be designed and measured.
2. **Implementation and performance.** A parallel implementation of persistent firing, measured against established Datalog engines on both interaction counts and wall time, is future work. The claimed full parallelism (Theorem 2.2) and order-independence (Theorem 2.3) make PΔ-nets a candidate substrate for massively parallel fixpoint evaluation; validating this empirically is open.
3. **Intermediate-state scheduling for cost models.** Theorem 6.1 establishes agreement of final derived sets; the intermediate states of Phase 1 are scheduler-dependent (§6.2). A cost model that predicts interaction counts under specific schedules, and an optimal scheduler for a given cost metric, are open.
4. **Richer value domains.** Extending the ground-term encoding (§3.5) and the expression layer (§8) to lattice-valued Datalog (Madsen, Yee & Lhoták 2016) and to aggregation would broaden applicability; the interaction between lattice joins and persistent firing is unexplored.

## 11. References


- Salvadori, D. A. R. (2025). Δ-Nets: Interaction-Based System for Optimal Parallel λ-Reduction. arXiv:2505.20314.
- Van Gelder, A., Ross, K. A., & Schlipf, J. S. (1991). The Well-Founded Semantics for General Logic Programs. *Journal of the ACM*, 38(3), 620–650.
- Van Gelder, A. (1989). The Alternating Fixpoint of Logic Programs with Negation. *PODS 1989*.
- Przymusinski, T. C. (1988). On the Declarative Semantics of Deductive Databases and Logic Programs. *Foundations of Deductive Databases and Logic Programming*.
- Chen, W., & Warren, D. S. (1996). Tabled Evaluation with Delaying for General Logic Programs. *Journal of the ACM*, 43(1), 20–74.

- Lévy, J.-J. (1978). Réductions Correctes et Optimales dans le Lambda-Calcul. Thèse de Doctorat d'État, Université Paris VII.
- Hăvărneanu, A. (2026). Universal Quantification for CUE: Semantics and Certification of Open Constraints. v9, 28 September 2026.
- Abiteboul, S., Hull, R., & Vianu, V. (1995). *Foundations of Databases*. Addison-Wesley. (Semi-naive evaluation, §3.3.)
- Lafont, Y. (1990). Interaction Nets. *POPL 1990*.
- Lafont, Y. (1997). Interaction Combinators. *Information and Computation*, 137(1), 69–101.
- Madsen, M., Yee, M., & Lhoták, O. (2016). From Datalog to Flix: A Declarative Language for Fixed Points on Lattices. *PLDI 2016*.
- Mokhov, A., Mitchell, N., & Peyton Jones, S. (2018). Build Systems à la Carte. *ICFP 2018*.
- Gupta, A., Mumick, I. S., & Subrahmanian, V. S. (1993). Maintenance of Materialized Views: Problems, Techniques, and Applications. *IEEE Data Engineering Bulletin*, 16(2), 3–18.
- McSherry, F., Murray, D. G., Isaacs, R., & Isard, M. (2013). Differential Dataflow. *CIDR 2013*.
- Staudt, M., & Jarke, M. (1996). Incremental Maintenance of Externally Materialized Views. *VLDB 1996*.
- Dalmau, V., & Kolaitis, P. G. (2008). Constraint Satisfaction, Bounded Treewidth, and Finite-Variable Logics. *CP 2002*.